

Multiagent Deep Reinforcement Learning With Demonstration Cloning for Target Localization

Ahmed Alagha¹, Rabeb Mizouni¹, Jamal Bentahar¹, Hadi Otrok², *Senior Member, IEEE*,
and Shakti Singh², *Member, IEEE*

Abstract—In target localization applications, readings from multiple sensing agents are processed to identify a target location. The localization systems using stationary sensors use data fusion methods to estimate the target location, whereas other systems use mobile sensing agents (UAVs, robots) to search the area for the target. However, such methods are designed for specific environments, and hence are deemed infeasible if the environment changes. For instance, the presence of walls increases the environment's complexity and affects the collected readings and the mobility of the agents. Recent works explored deep reinforcement learning (DRL) as an efficient and adaptable approach to tackle the target search problem. However, such methods are either designed for single-agent systems or for noncomplex environments. This work proposes two novel multiagent DRL models for target localization through search in complex environments. The first model utilizes proximal policy optimization, convolutional neural networks, Convolutional AutoEncoders to create embeddings, and a shaped reward function using breadth first search to obtain cooperative agents that achieve fast localization at low cost. The second model improves the first model in terms of computational complexity by replacing the shaped reward with a simple sparse reward, subject to the availability of Expert Demonstrations. Expert demonstrations are used in Demonstration Cloning, a novel method that utilizes demonstrations to guide the learning of new agents. The proposed models are tested on a scenario of radioactive target localization, and benchmarked with existing methods, showing efficacy in terms of localization time and cost, in addition to learning speed and stability.

Index Terms—Imitation learning (IL), multiagent deep reinforcement learning (MDRL), proximal policy optimization (PPO), reward shaping, target localization.

Manuscript received 27 January 2023; revised 9 February 2023; accepted 25 March 2023. Date of publication 28 March 2023; date of current version 25 July 2023. This work was supported in part by the Fonds de Recherche en Nature et Technologies (FRQNT); in part by the Natural Sciences and Engineering Research Council of Canada (NSERC); and in part by the Department of National Defense [Innovation for Defence Excellence and Security (IDEaS)]. (Corresponding author: Ahmed Alagha.)

Ahmed Alagha is with the Concordia Institute for Information Systems Engineering, Concordia University, Montreal, QC H3G 1M8, Canada (e-mail: ahmed.alagha@mail.concordia.ca).

Rabeb Mizouni, Hadi Otrok, and Shakti Singh are with the Department of Electrical Engineering and Computer Science and the Center of Cyber Physical Systems, Khalifa University, Abu Dhabi, UAE (e-mail: rabeb.mizouni@ku.ac.ae; hadi.otrok@ku.ac.ae; shakti.singh@ku.ac.ae).

Jamal Bentahar is with the Concordia Institute for Information Systems Engineering, Concordia University, Montreal, QC H3G 1M8, Canada, and also with the Department of Electrical Engineering and Computer Science, Khalifa University, Abu Dhabi, UAE (e-mail: jamal.bentahar@concordia.ca; jamal.bentahar@ku.ac.ae).

Digital Object Identifier 10.1109/IJOT.2023.3262663

I. INTRODUCTION

THE rapid development of intelligent robots, equipped with sensors and computational capabilities, has enabled their use in several applications, such as environmental monitoring and target localization [1], [2]. Target localization is a problem in which the location of a certain target is to be identified using sensory data readings collected by multiple sensing agents. Such a problem is of interest in applications, including search and rescue missions [3], fire detection [4], object tracking [5], and radiation monitoring [2], [6]. In such applications, the existence of obstacles and walls increases the complexity of the environment, and hence of the localization problem, as they not only affect the movement of the sensing agents but also their collected data readings [7].

Target localization problems can be categorized into two subsets: 1) target estimation and 2) target search. Traditional localization methods rely on centralized data fusion of sensory readings from multiple sensing nodes, in order to estimate the target location. Such methods include Bayesian methods [8], Copula theory [9], and maximum-likelihood estimate [10]. However, these methods suffer from uncertainty when localizing the target, since they rely on the use of previously placed stationary sensing nodes that only give an estimation to the target location. In many cases, the lack of informative readings motivates the need for mobile sensing nodes that search the area for the target. Alternative approaches are target search methods that rely on mobile sensing agents, such as robots and UAVs, to search the Area of Interest (AoI) for the target by surveying some predefined or data-driven paths [8], [11], [12], [13]. The problem here is similar to path-finding problems, where the aim is to find the shortest/fastest path to a certain target, with the major difference that the target location is unknown in target search and localization problems, which makes the problem significantly more complex. In methods relying on predefined paths, the agents follow a search path that is designed for the specific environment of interest. In data-driven methods, the agents follow a path based on the data collected, aiming to move toward regions with higher readings. These methods prove efficient in simple environments but suffer from adaptability issues. In other words, they are designed for a specific environment, and hence are deemed unusable if the environment changes, thus requiring intervention from the designer to remodel such methods for new environments.

Recent methods have explored the use of deep reinforcement learning (DRL) in tackling the target localization

problem. DRL helps in acquiring intelligent sensing agents through learning models that are adaptable to different environments. Here, during the training phase, which mainly happens offline in simulations, an agent learns a policy for decision making based on its experience in the environment, guided by a numerical reward signal that the agent aims to maximize [14]. Such a reward signal could be sparse, i.e., it rarely appears in the state space of the problem, or shaped, where the reward appears more frequently in the state space [15]. The learning could be off-policy, where old experiences could be used to update the current policy, or on-policy, where the most recent experiences are required for the update process. The learnt policy is then exploited in the inference/deployment phase to tackle the given task. In DRL, agents' policies are represented by deep neural networks, trained to maximize the cumulative reward [16]. Multiagent DRL (MDRL) is a paradigm that extends DRL to include multiple agents, which could be cooperating or competing, in the same environment. MDRL has been proposed to tackle problems in several domains, including video games [17], autonomous driving [18], and robot swarms [19], to name a few. Recent works [7], [20] have approached the target localization problem in single-agent DRL settings but prove inefficient when extrapolated in multiagent settings. This is mainly due to the complexity overhead challenge (i.e., curse of dimensionality [16]) resulting from having multiple agents in MDRL. Another work [6] addressed the problem in multiagent settings, but without considering complex environments with walls, which significantly complicates the problem. In summary, the drawbacks in existing literature, which are to be tackled in this work, are as follows.

- 1) Traditional target localization methods are not adaptable to varying environment complexities.
- 2) Single-agent DRL methods for target localization cannot be directly extrapolated into MDRL due to the curse of dimensionality.
- 3) Existing MDRL-based methods for target localization do not consider complex environments with obstacles.

The contributions of this work are embedded in two novel MDRL models that tackle the target localization through search in complex environments. The main goal is to utilize agents' observations, i.e., data readings and information about the other agents and the environment, to decide on movement actions to optimally search the area for the target. Initially, a model is designed which utilizes proximal policy optimization (PPO) [21] with convolutional neural networks (CNNs) to model, train, and optimize the agents' policies. Agents' observations are modeled as 2-D heatmaps capturing information about their own, and the locations of other agents, in addition to the collected readings and the distribution of walls in the environment. A shaped team-based reward function, using breadth first search (BFS), is designed to guide the agents in *cooperating* to localize the target. Convolutional autoencoders (CAEs) are used to create embeddings that efficiently represent the walls in the environment. A centralized-learning and decentralized-execution (CLDE) method is used to train the agents, which helps in tackling the curse of dimensionality issue in MDRL. This model is then further improved in a second model by replacing the shaped reward function

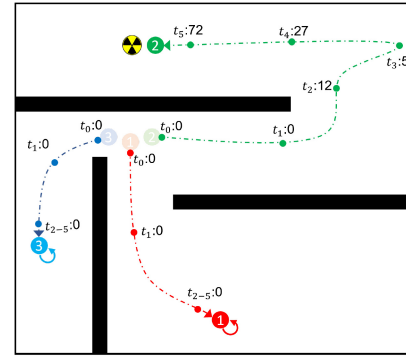


Fig. 1. Representation of the target localization problem using a radioactive environment with three agents and three walls. The readings collected by each agent (in photon counts per minute), at each time step t_i , are given along the paths.

with a sparse reward to speed up the learning. This is combined with demonstration cloning (DC), a novel approach that utilizes demonstrations from expert or semi-expert agents, similarly to imitation learning (IL), to guide the RL agents in the learning process, aiming to achieve faster and more resilient learning. This results in two efficient methods that could be used depending on the availability of expert demonstrations. In summary, the contributions of this work are as follows.

- 1) The formulation of the multiagent target localization problem in complex environments using MDRL.
- 2) The modeling of the agents' observations as 2-D heatmaps, and using CAEs to create embeddings for walls. Observations are fed to CNNs that specify the actions, which are optimized using PPO, and guided by a shaped reward function that is designed using BFS.
- 3) The design of DC; a method that utilizes expert experiences in guiding agents into faster and more resilient on-policy MDRL in the presence of sparse rewards.

The proposed models are evaluated for the scenario of radioactive target localization, where the agents are to localize a radioactive source of a given intensity by progressively searching the AoI based on the collected observations. The models are analyzed in terms of the learning performance, showing scalability and adaptability to variations of the environment, with human-like performance developed in terms of cooperation. The performance of the resulting agents is compared against existing target localization and MDRL benchmarks, varying from traditional target localization methods [8], [12], [13] to MDRL-based target localization methods [6], [19].

II. PROBLEM DEFINITION AND MOTIVATION

In this section, the target localization problem in complex environments is defined, and a motivational scenario is given to highlight the drawbacks in existing works.

A. Target Localization Problem

The target localization problem tackled in this work is represented in Fig. 1, and is defined as follows: given N agents and a certain AoI within which an unknown target is located, the agents are to cooperate to search the area and find this target. An agent should use the collected observations, including the

data readings and information about the environment and other agents, to take movement actions in the environment aiming to find the target. The observations are collected in a progressive manner and are updated after each step in the environment. Data readings might not always be sufficient, which requires the agents to cooperate in exploring the environment to collect more readings, which can be then used for better decision making. It is assumed that agents are capable of storing their history in terms of data readings and locations visited. Agents are also capable of communicating with each other to share information. Obstacles in the environment increase the complexity of the problem, as they attenuate the signal from the source (i.e., the radiation) and affect the mobility of the agents.

The desired behavior in such environments is for agents to quickly reach the unknown target location while cooperating to preserve resources. The idea is, even though the target location is unknown, the data readings induced by the target should help the agents take the right actions that lead to exploring the environment in a way that leads to quick localization. The agents should be able to tackle the localization problem for any target location and any initial distribution of agents and walls. Initially, a coordinated exploration is needed for the agents to collect some readings and build an intuition about the possible target locations. Cooperation should also be seen in resource management, as agents should only attempt to contribute to the task (by searching) if they can add benefit to the team. These desired behaviors are illustrated in Fig. 1, which presents a simplified scenario. Initially ($t_0 - t_1$), the three agents have similar low readings, and hence the ideal situation would be to coordinate and split to cover and explore a wider area. At t_2 , and given the readings and location histories of the three agents, it is evident that agent 2 has come closer to the target, and hence agents 1 and 2 choose to stay idle to preserve resources. At the same time step, agent 2 is uncertain as to which direction to move, and hence chooses to explore the area to the right, before realizing that the better action is to move left. This example summarizes three desired behaviors in the agents: 1) to cooperate and explore when data readings are insufficient; 2) to incorporate the history of readings/locations into the decision-making process with the aim of moving toward areas with potentially higher readings; and 3) to preserve resources when the agents cannot be beneficial to the target localization task. This is while considering the existence of obstacles/walls that increase the complexity of the problem. These behaviors are to be developed by the agents themselves through rewarded interaction with the environment using the proposed MDRL models.

B. Motivational Scenario

This section highlights the drawbacks in existing DRL methods tackling the target localization problem. The proposed methods in [6], [7], and [20] are tested in an environment of three agents and three walls, aiming to localize an unknown radioactive target, similar to the scenario shown in Fig. 1. An episode of the problem lasts 100 timesteps, where in each time step an agent has nine possible actions: to move in one of eight

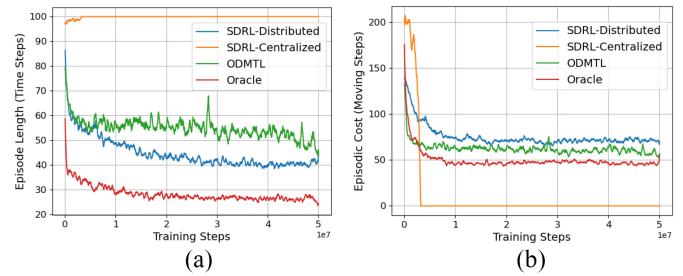


Fig. 2. Learning performance in terms of (a) localization time and (b) localization cost of 3 existing methods. Time and cost are given as a function of the learning steps, i.e., the number of steps executed in the environment during the DRL process.

cardinal directions or to stay idle. Single-agent DRL works in [7] and [20] are directly extrapolated to multiagent settings in two methods. In the first method (namely, SDRL-distributed), each agent acts on its own as if no other agents exist. In the second method (SDRL-centralized), a single centralized policy network takes as input the observations of all agents (locations, readings, and walls), and produces three actions, one for each agent. The MDRL work in [6] (namely, ODMTL) uses distributed execution, where each agent has its own policy and acts based on its observations, which include information about other agents for cooperation. This work does not consider the existence of walls, and hence they are masked out from the agents' observations. Fig. 2 shows the learning performance of the three methods, in terms of localization time and cost. Time here reflects the number of timesteps from the beginning of a localization episode until the target is localized (if the episode limit is not reached), while cost reflects the total number of moving steps (cost of moving). An oracle representing an ideal policy is shown for reference, which will be later discussed in Section IV along with details about the observations and the learning process.

As shown in Fig. 2, the performance of SDRL-centralized is significantly poor, as the learning fails to converge. Here, agents fail to learn how to localize, and decide to constantly stay idle, which results in 0 cost but failure to find the target (episodes reach the limit). This is mainly due to the curse of dimensionality introduced by using multiple agents, which cannot be handled by a single centralized policy. Specifically, in this case, the policy has 729 possible combinations of actions at each time step, out of which the best combination is to be chosen. On the other hand, SDRL-distributed shows better convergence, as agents learn to localize the target. However, when compared to the oracle, it can be seen that the localization cost is notably higher. This is because the agents have no form of cooperation, since each agent acts independently without considering other agents. Finally, while ODMTL provides a distributed method where agents are aware of each other and capable of cooperating, the ignorance of walls makes this method fall behind the oracle, since walls significantly affect the collected readings (due to attenuation) and the mobility of agents. This work aims to circumvent the issues of curse of dimensionality, agents' cooperation, and fast learning in complex environments, using methods based on team-based shaped rewards,

centralized-learning with distributed-execution, CAEs, and learning from demonstrations.

III. RELATED WORK

Target localization is approached in the literature in several ways, with the common aim of quickly and efficiently finding the target location. Traditional works design and build mathematical data fusion models that integrate sensory data readings from multiple sensing nodes to estimate the target location. The works in [10] and [22] use maximum-likelihood estimation (MLE) and Inverse Square Law as mathematical methods to estimate the location of a radioactive source. Bayesian methods are used in [8] to estimate the target location, where data readings are iteratively fused over time to update the prior belief about the target location. The works in [23] and [24] utilize Time Difference of Arrival (TDoA) and Direction of Arrival (DoA) to localize sound events. Some works complement the above works by focusing on the placement and selection of sensing nodes, aiming to optimize resources while quickly and efficiently localizing the target. The works in [8] and [25] use greedy and genetic methods to optimize the placement of sensing nodes, with the aim of maximizing the localization accuracy and minimizing the localization time. Active node selection is tackled in [2], [26], [27], and [28], where data-driven mechanisms are proposed to select and activate informative nodes for localization tasks, while considering nodes' attributes, such as the residual energy, data confidence, and area coverage. A recent work in [29] proposes a RL approach using Q -Learning to tackle the sensor selection problem for target localization in Internet of Things (IoT) systems. While the above works prove efficient in managing resources, they only provide location estimates of the target, since no agents are physically searching for the target. Additionally, as the complexity of the environment increases (i.e., with obstacles), these solutions become infeasible, since obstacles significantly affect the sensor readings collected by the nodes, which would require further remodeling and supervision of the proposed methods for different environments.

Other proposals tackling the target localization problem focus on designing survey paths that are either predefined or data driven, which are followed by mobile sensing agents to search for the target. The traditional uniform search method [13] is used in many works due to its simplicity, where a uniform survey path is predefined. A set of circular path planning strategies is proposed in [11] based on the directional characteristics of sensors. In [12], a data-driven surveying approach using Bayesian methods is proposed, where probability density functions (PDFs) about the target location are obtained from the individual agents and combined to decide what actions to take next. Recent works approached the target search problem using DRL, where the sensing agents are trained (mostly offline in simulations) before being deployed in an actual environment. The work in [7] uses Double Q -Learning with CNNs to train a single agent in a radioactive environment with obstacles, aiming to find the target location. In this work, obstacles are modeled as 2-D maps

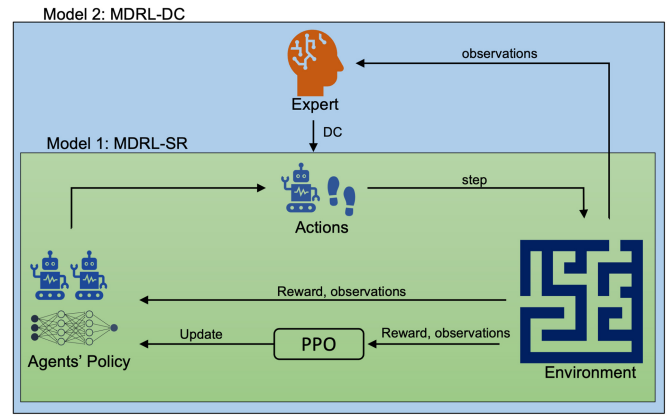


Fig. 3. General overview of the proposed models.

that are fed to the CNNs policy networks as part of the agent's observations. A similar work in [20] uses PPO combined with particle filter gated recurrent units in single-agent settings. This work considers obstacles using proximity sensors attached to the agents, which helps the agents in avoiding obstacles. These works, however, suffer from significant scalability issues when extrapolated into multiagent settings. In multiagent settings, the work in [6] uses PPO with CNNs, combined with team-based rewards, to tackle the target localization problem in a distributed manner, while ensuring cooperation between the sensing agents. However, this work does not consider complex environments with walls.

IV. PROPOSED SYSTEM

This section presents the two proposed MDRL models for target localization tasks. Fig. 3 gives a general overview of the two proposed models. In the first model [MDRL with shaped reward (MDRL-SRs)], agents' policies translate observations into actions that are executed in the environment to obtain a shaped reward and the next observations, which are used for the following step. PPO incorporates the collected rewards to update the agents' policies. Given the availability of expert demonstrations, the second model [MDRL with DC (MDRL-DC)] builds on the first model by using DC in guiding the agents toward collecting better experiences in the environment. The proposed models share the same observation and action spaces, as well as the same policy structure, but differ in the reward function and the learning process.

A. MDRL Formulation and Policy Optimization

In single-agent RL settings, the environment is usually formulated as a Markov decision process (MDP), which characterizes the decision making of an agent with full observability of the system state [16]. One way to generalize MDP to account for multiple agents is using Markov Games [30], which redefine the game-theoretic stochastic games in the RL context. In target localization problems, the state space of the environment is defined by the target location and the distribution of agents and walls throughout the area, i.e., each target location combined with a distribution of agents and walls give a unique state. Since the target location is unknown and cannot

be observed by the agents, the problem is modeled as a partially observable Markov game (POMG). Here, the state space is substituted with an observation space, which incorporates the information the agents can observe in the environment. This includes the locations of other agents, the environment walls, and the collected data readings that, if sufficient, can be used to infer the target location. The resultant observation space is very large, and it is impossible to train on all combinations in this space, as in all RL problems. Hence, deep neural networks (function approximators) are used in the training (to be discussed in Section IV-D), as they help in *generalizing* from seen to unseen observations (i.e., training on a small subset of the observation space helps in tackling observations that were not seen during the training). POMG can be defined as a tuple $(\mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- 1) $\mathcal{N} = 1, \dots, N$ denotes the finite set of agents, where $N \geq 1$;
- 2) $\mathcal{S}$ denotes the finite set of states;
- 3) $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2 \times \dots \times \mathcal{A}_N$ is the set of joint actions, where $\mathcal{A}_i$ denotes the finite set of actions available for agent i , and $\mathbf{a} = (a_1, \dots, a_N)$ denotes a joint action;
- 4) $\mathcal{O} = \mathcal{O}_1 \times \mathcal{O}_2 \times \dots \times \mathcal{O}_N$ is the set of joint observations, where $\mathcal{O}_i$ denotes the finite set of observations for agent i , and $\mathbf{o} = (o_1, \dots, o_N)$ denotes a joint observation;
- 5) $\mathcal{P}$ is a set of Markovian state transition and observation probabilities, where $\mathcal{P}(s', \mathbf{o}|s, \mathbf{a})$ denotes the probability that taking joint action $\mathbf{a}$ in state s results in a transition to state s' and joint observation $\mathbf{o}$;
- 6) $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a reward function;
- 7) $\gamma \in [0, 1]$ is the discount factor for the tradeoff between instantaneous and future rewards.

For the target localization problem, the game unfolds over a finite sequence of steps. At every step, each agent i analyzes its observation $o_i \in \mathcal{O}_i$ and takes action $a_i \in \mathcal{A}_i$ based on a policy $\pi_i : \mathcal{O}_i \times \mathcal{A}_i \rightarrow [0, 1]$ and receives a reward r_i . When looking at the full picture, all agents simultaneously select an action $\mathbf{a} = (a_1, \dots, a_N)$ and receive a reward and observation $\mathbf{o} = (o_1, \dots, o_N)$. The objective for each agent is to maximize the expected sum of rewards it receives during the game. After each step, the collected observations are progressively updated with new data readings and new agents' locations, which can be used in the following step.

In this work, PPO [21] is used to update the agents' policies. PPO is a state-of-the-art policy gradient method for RL that uses the actor-critic structure. PPO is used in this work due to its simplicity, low computational complexity, and balance between sample efficiency and wall-clock time. Here, a policy π_θ , parameterized by θ , is to be optimized with the objective of maximizing the cumulative reward throughout the episode. In the actor-critic structure, the policy maps the current observations to a probability distribution over the possible actions, while the estimated value function (critic) is used to criticize the actions made by the actor. PPO uses the current experiences, combined with the critique, and tries to take the biggest improvement step to update the current policy, without moving far from it. This tackles the issue of destructively large policy updates in traditional PG methods, which could cause considerable loss of performance. PPO optimizes a clipped policy

surrogate, which is given as

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (1)$$

where ε is a clipping hyperparameter. In this function, $r_t(\theta) = ([\pi_\theta(a_t|s_t)]/[\pi_{\theta_{\text{old}}}(a_t|s_t)])$ is the probability ratio of taking certain actions between the old and the current policy. $\hat{A}_t$ corresponds to an estimator of the advantage function; a function that measures how good a certain action is in a given state. In this work, we use generalized advantage estimate (GAE) [31] to estimate $\hat{A}_t$. For each update, PPO uses a fixed length trajectory H , known as the horizon length, which runs the policy for H time steps to collect experiences. The estimator of the advantage function is then given as

$$\begin{aligned} \hat{A}_t^{\text{GAE}(\gamma, \lambda)} &= \sum_{l=0}^H (\gamma \lambda)^l \delta_{t+l}, \quad \delta_{t+l} \\ &:= r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l}) \end{aligned} \quad (2)$$

where δ_{t+l} is the temporal difference (TD) residual, $\gamma, \lambda \in [0, 1]$ are discount factors, and $V(s_t)$ is the value predicted by the critic.

The objective function in (1) is combined with two other terms, $L_t^{\text{VF}}(\theta)$ and $S[\pi_\theta](s_t)$, to give the following final objective function that is maximized at each iteration:

$$L^{\text{CLIP+VF+S}}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 S[\pi_\theta](s_t) \right] \quad (3)$$

where c_1 and c_2 are coefficients. In this function, $L_t^{\text{VF}} = (V_\theta(s_t) - V_t^{\text{targ}})^2$ denotes the squared-error loss (value function error), while S denotes an entropy bonus that ensures sufficient exploration.

B. Observation Space

The process of localizing the target requires knowledge of current and historical data readings, in addition to the locations where these readings were collected. For this reason, the agents' observations are modeled as 2-D maps containing information related to the readings collected, in addition to the agents' locations. To obtain this, the AoI is represented as a grid of size $m \times n$, and the observations are modeled as a stack of 2-D maps, where each of them has a size of $m \times n$. The choices of m and n affect the details of the representation, where higher values allow representing finer details at the cost of higher computational complexity. The grid dimensions are not to be confused with the actual dimensions of the AoI. The dimensions of the AoI could be in meters or kilometers, depending on the application, which would be divided into the corresponding $m \times n$ grid. Each of the 2-D maps is normalized before being fed to the neural networks, which is a standard practice in deep learning that ensures faster convergence [32]. Here, the normalization of a 2-D map is done by dividing all of its elements by the maximum pixel value, resulting in values between 0 and 1. Fig. 4 shows the five normalized observations collected by each agent at a given timestep t , which are described as follows.

- 1) *Location Map*: A 2-D map showing the agent's own location.

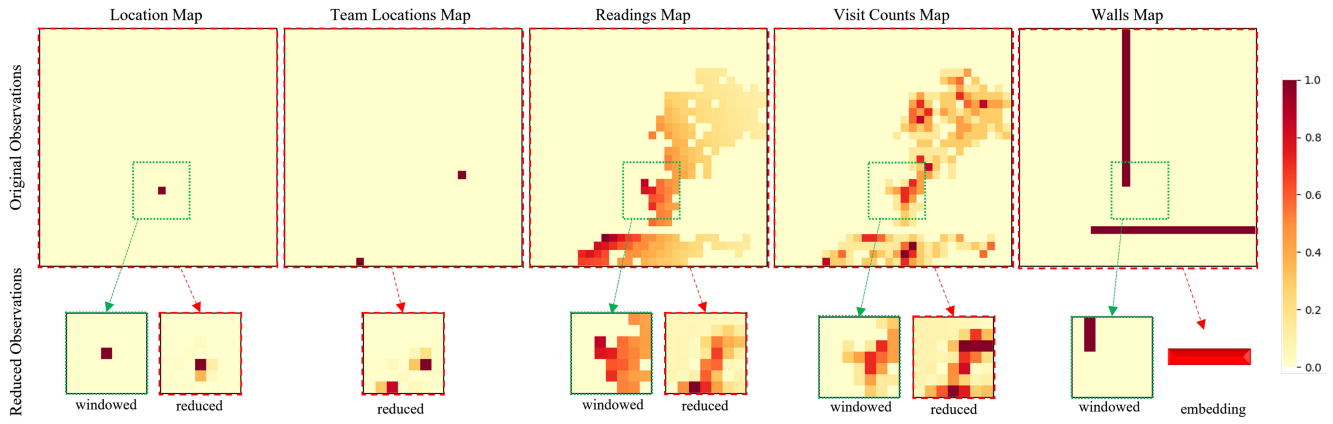


Fig. 4. Set of observations collected by an agent, in a team of three agents, in an AoI of size $1 \text{ km} \times 1 \text{ km}$ and grid dimensions of 30×30 . The original observations are shown at the top, while the reduced observations are shown at the bottom. Amongst the reduced observations, local observations are highlighted in green (windowed), while the global observations are highlighted in red.

- 2) *Team Locations Map*: A 2-D map showing the locations of all the other $N - 1$ agents.
- 3) *Readings Map*: A 2-D map showing the last data reading collected in each grid element by any of the agents. Grid elements that have not been visited are assigned a data reading of 0.
- 4) *Visit Counts Map*: A 2-D map that keeps track of the frequency at which each grid element has been visited by any of the agents.
- 5) *Walls Map*: A 2-D map showing the walls in the area.

Modeling the observations as 2-D maps helps in preserving information. This way, the information could be accumulated in a progressive manner on the same 2-D maps as the target localization process carries on. Additionally, agents' locations and the readings collected are spatially correlated in target localization tasks, and such spatial features can be maintained and analyzed when the observations are modeled as 2-D maps.

In the target localization problem, each agent bases its actions solely on the aforementioned observations. In these observations, the readings map is used to guide the agents toward the target location, where the agents are expected to follow the paths leading to higher readings. The readings map is built in a progressive manner, where the map initially could have insufficient and uninformative readings (due to the agents being far from the target or due to signal attenuation because of walls). At each step, the agents take actions and update the readings map with the collected readings, after which it can be better used to guide the agents to reach the target. With the existence of walls in the area, the agents could initially need to move away from the target (and hence move toward low data readings), as seen in Fig. 1 for agent 2. This could be achieved when incorporating the readings and walls maps together. In cases where the agents fail to collect good readings initially, they are expected to explore unvisited areas, which is enabled through keeping track of the visited locations in the visit counts map. To help with cooperation and coordination between the team members, the Team Locations map is used. The agents are assumed to have full communication, and hence are capable of sharing their own locations and readings.

Before feeding the observations to the actor and critic networks, they undergo preprocessing steps to reduce dimensionality while maintaining the important features. This helps with speeding up the learning process, as the number of trainable parameters is reduced. To do so, the five aforementioned original observations are converted into the nine reduced observations, as shown in Fig. 4. Each of the first eight reduced observations has a size of $w \times w$, where $m, n > w > 1$ and w is odd. w is a hyperparameter, where smaller values lead to less trainable parameters in the policy network (and hence reduced learning complexity) but also lead to higher loss of information, when compared to higher values. The main goal is to obtain reduced observations that still maintain the essential information from the original observations. To achieve this, the reduced observations are divided into two sets: 1) local and 2) global observations. The local observations help an agent make local decisions regarding the areas surrounding them. Such decisions include overcoming nearby obstacles and moving toward unvisited areas or areas with higher readings within the small region around them. On the other hand, the global observations give the agent a summary of the information in the AoI, which helps in coordinating with other agents and planning ahead.

The local observations capture the information surrounding the agent, in a $w \times w$ map centered around the agent's current location, and are given as follows at step t .

- 1) *Windowed Location Map*: A 2-D map showing the agent's location.
- 2) *Windowed Readings Map*: A 2-D map showing the latest data readings collected in a small region around the agent.
- 3) *Windowed Visit Counts Map*: A 2-D map showing the frequency of the visited areas around the agent.
- 4) *Windowed Walls Map*: A 2-D map showing the walls near the agent.

The global observations have a size of $w \times w$ and are as follows for a given agent, at timestep t .

- 1) *Reduced Location Map*: A 2-D map obtained by down-sampling the agent's original location map.

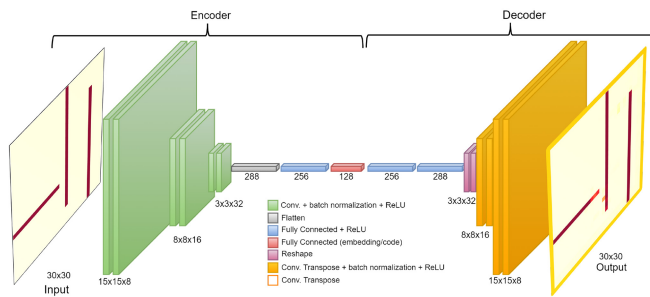


Fig. 5. Architecture used for the CAE. The CAE is pretrained using a data set of walls, after which the encoder is used to embed the wall maps as part of the proposed MDRL models.

- 2) *Reduced Team Locations Map*: A 2-D map obtained by downsampling the agent’s original team locations map.
- 3) *Reduced Readings Map*: A 2-D map obtained by downsampling the agent’s original readings map.
- 4) *Reduced Visit Counts Map*: A 2-D map obtained by downsampling the agent’s original visit counts map.
- 5) *Walls Embedding*: An embedding of size $1 \times d$ obtained through encoding the agent’s original walls map using a pretrained CAE.

The first four global observations are obtained through downsampling using bilinear interpolation. While generic downsampling is effective for these observations, the loss of information would be notable when applied to the walls map. This is mainly because, in some cases, the agents need to navigate through narrow areas/entrances, and information may be lost if traditional downsampling techniques are used. To overcome this, CAEs are used to embed the walls map into an embedding (1-D vector) of reduced dimensions. The structure of the used CAE is shown in Fig. 5. Given a data set of walls, which is synthetically created using variations of wall placements and wall lengths, the CAE is trained to embed the walls map into a vector of length d . A typical autoencoder consists of two components: 1) an encoder and 2) a decoder. The encoder maps the input to an embedding (a code), and the decoder aims to reconstruct the original input from the code. Typically, the training loss is based on the difference between the input and the reconstructed map, which is to be minimized, indicating that the code has enough information to reconstruct the original image from. Once an autoencoder is trained, the encoder is then used to create embeddings for the wall maps observations.

C. Action Space

In the proposed models, the agents take discrete actions in the AoI: to either move in a certain direction or stay idle. The choice of staying idle helps in preserving resources, while moving affects the contribution the agent has toward the target localization task. The direction of movement is discretized into B possible directions $\{1, 2, \dots, b_i, \dots, B\}$, where the direction (or angle) of movement is given as

$$\theta = 2\pi \frac{b_i}{B}. \quad (4)$$

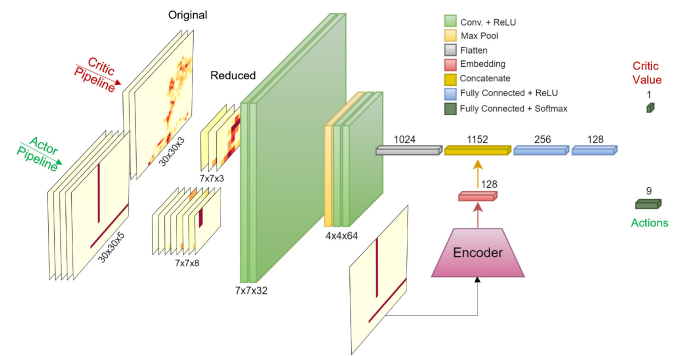


Fig. 6. Actor and critic networks (CNN) used in the proposed models.

B here is a hyperparameter that could be determined based on the desired details and the available computational capabilities. Higher values of B allow more mobility freedom for the agents but also increase the learning overhead as the policy network would have more trainable parameters. In this work, the agent has nine possible actions, being eight possible cardinal directions ($B = 8$) and one action for staying idle. This has been proved sufficient in this work, as it covers the horizontal, vertical, and diagonal directions. It is assumed that all agents have the same fixed speed. At each time step, certain actions may be invalid (such as moving into a wall or outside the boundaries of the AoI). In such cases, these actions are masked out, and the agent has to choose from the set of viable actions. This is applied during the learning and the inference processes. Additionally, during learning and inference, if all agents choose to stay idle for three successive timesteps, the idle action is masked out for the next step for all agents, to encourage exploration and to speed-up the learning.

D. Actor and Critic Networks

The proposed models use an actor-critic structure in the learning process, optimized by PPO. The actor (policy) network translates input observations into actions, while the critic network is used to estimate the value function. Since the observations in this work are modeled as 2-D maps, CNNs are used to represent the actor and critic. CNNs help in capturing spatial features in input maps, which are common in aforementioned observations

The structures of the actor and critic networks are shown in Fig. 6. This architecture is similar to the LeNet-5 architecture [33] which is widely used in image analysis tasks. At each time step, for a given agent, the actor network takes as input the nine reduced observations, shown previously in Fig. 4, and produces $B+1$ actions. The first eight observations (2-D maps) are fed through convolutional and Max-Pooling layers to extract features. The resultant maps are flattened and concatenated with walls embedding, and fed to the fully connected layers. The 1-D walls embedding does not contain any spatial information, and hence is not passed through the convolutional layers. A SoftMax function is used at the last layer so that the outputs correspond to a probability distribution over the possible actions. The proposed models in this work follow a CLDE [34] method. Each agent has the same

copy of the actor network during learning and execution, and acts based on its own observations in a decentralized manner. However, during the learning process, the critic network has access to all observations. A centralized critic helps in tackling the nonstationarity issue, which is caused due to the environment being influenced by other agents from a single agent's perspective [35]. This is viable since the critic is not needed and emitted during the execution/deployment stage. For this reason, the critic CNN takes as input the following four observations.

- 1) *Reduced All-Locations Map*: A downsampled version of a 2-D map showing the locations of all agents in the AoI.
- 2) *Reduced Readings Map*.
- 3) *Reduced Visit Counts Map*.
- 4) The walls embedding.

These observations are sufficient for the critic to give a value to the current state of the environment, which is a singular output given by the critic network. The critic considers the global observations and gives a value representing how good the current state is. This value is used in the PPO process to update the networks, as discussed later in Section IV-E. The input 2-D maps to the neural networks are normalized beforehand, which is a typical process when using CNNs.

E. Reward Function and Learning Process

This section presents and discusses the two proposed models. The two models share the same previously presented observation and actions spaces, and the same network architectures, but differ in terms of the reward function and the learning process.

1) *Model 1 (MDRL With Shaped Rewards)*: This section presents the first model tackling the target localization problem in complex environments.

Reward Function: This model uses a *shaped* reward function to guide the learning process. To push the agents to cooperate, the reward function is also *team-based* (joint) [34]. Following the actions taken individually by the agents, the environment releases a shared (equal) reward to all the agents based on the joint actions. This helps the team in realizing the advantage of taking actions that benefit the team.

At a given time step, and given N agents with the joint action $\mathbf{a}^t$ at step t , the reward function is given as

$$R_t = \begin{cases} -m + 1, & \text{if } \min(M_t) < \min(M_{t-1}) \\ -m - 1, & \text{otherwise} \end{cases} \quad (5)$$

where M is the set of distances between the agents and the target, and m be the number of agents that took a moving action (i.e., did not choose to stay idle). The $(-m)$ term represents the cost of movement, being -1 for each agent that moves. The second term (± 1) assesses if the agents have gotten closer to the target or not. At step t , the team is considered to have gotten closer to the target if the closest agent (or one of the closest agents) to the target at step $t - 1$ gets closer to the target at step t , i.e., if $\min(M_t) < \min(M_{t-1})$. For example, considering the scenario in Fig. 1, the team would get a reward of -2 at t_0 (3 agents moved and the team got closer to the target), a reward of -2 at t_2 (1 agent moved and the team did not get closer to the target), and a reward of 0 at

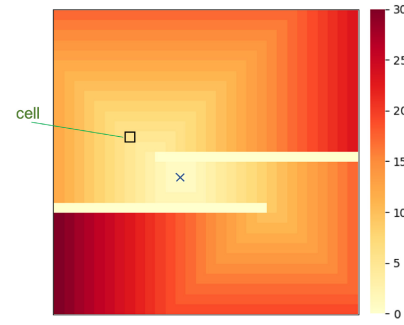


Fig. 7. Example of the distance map obtained at the beginning of an episode. Each cell contains a value representing the shortest distance (in number of steps) between the cell and the target (marked with $\times$).

t_3 (one agent moved and the team got closer to the target). It is worth mentioning that the reward function is used only during the training stage (offline in simulations) as a feedback mechanism to assess the actions taken by the agents. While the reward function is based on the distance to the target, the agents solely act based on their observations, with no knowledge of the target location. Once the training is completed, i.e., during the inference/deployment stage, the reward function is omitted, and the agents exploit whatever policy they have already learnt. Distance-based reward functions are common in similar works, as in [6], [7], and [20].

Using this shaped reward, the agents get feedback throughout the episode, and not only when the target is localized, which helps in learning even if the episode terminates without finding the target. The agents are motivated to finish the localization quickly, to accumulate less moving cost. Additionally, the agents are motivated to cooperate in order to move only when necessary (i.e., when they can contribute to the task), which helps in performing the task efficiently while managing resources.

To compute the distance between the agents and the target (during learning), conventional methods like Euclidean/Manhattan distance cannot be used due to the existence of walls. To tackle this, the proposed model uses the $m \times n$ gridded AoI presented in Section IV-B, along with the BFS method to compute the shortest distance between each agent and the target. BFS starts from an initial node, and gradually explores paths starting from this node until the goal is reached. A path here is given as a set of grid elements that lead the agent to the target. It is computationally expensive to perform BFS at each time step, for each agent. To expedite the process, at the beginning of each episode, a *distance map* is built by computing the distance between the target and all the grid elements in the AoI grid. This can be done with a single BFS process by setting the starting node as the target's location and ignoring the goal check in the BFS algorithm. As a result, BFS would continue to explore all nodes, and hence compute the distance to all the grid elements. A sample of the distance map is shown in Fig. 7.

Learning Process: This model follows a CLDE [34] method, with PPO to train the actor/critic networks. The training process is summarized in Algorithm 1. Each episode starts with a reset that returns the initial observations for each agent

Algorithm 1 Training Process for the MDRL-SR Model

Input: Initial actor and critic networks

```

1: while Step ≤ MaxNumOfSteps do:
2:    $\mathbf{o}^0 = \text{Env\_Reset}()$  #initial observations
3:   for  $i = 0, 1, 2, \dots, \text{Episode\_Length}$ :
4:     for  $j = 1, 2, \dots, \text{Team\_Size}$ :
5:        $a_j^i = \text{sample}(\text{actor}(\mathbf{o}_j^i))$  #sample an action from the actor output
6:     end for
7:      $\mathbf{a}^i = [a_1^i, a_2^i, \dots]$ 
8:      $\mathbf{o}^{i+1}, r^i, d^i = \text{Env\_Step}(\mathbf{a}^i)$ 
9:     Store  $\mathbf{o}^{i+1}$ ,  $\mathbf{a}^i$ , and  $r^i$  in their corresponding buffers
10:    Step = Step + 1
11:    if Step %  $H == 0$  then: #PPO update
12:      Compute the advantage estimate  $\hat{A}$  using the
        rewards and the value function  $V$  (critic)
13:      Update the actor (policy) and critic networks using
        the loss  $\mathcal{L}^{CLIP+VF+S}(\theta)$ 
14:      Empty all buffers
15:    end if
16:    if  $d^i == 1$  then break #if episode terminates
17:  end for
18: end while

```

($\mathbf{o}^0 = (o_1^0, \dots, o_N^0)$). Each agent gets a copy of the actor network, which takes a set of observations for agent j and returns an action a_j . Each of the agents acts in the environment and receives a set of new observations, a shared reward (the same for all agents), and a termination flag (d) indicating whether the episode is finished or not. These items are stored in their corresponding buffers, and later used (after H timesteps) to update the actor and critic networks. H here represents the horizon, i.e., the number of steps in the environment after which a PPO update occurs. During deployment/inference, only the actor network is used, where each agent gets a copy of the network and acts in a distributed manner.

This model proves efficient in obtaining agents that are capable of localizing the target in complex environments, as will be shown in Section V. The proposed reward function helps in pushing the agents to quickly localize the target, while cooperating to manage resources. However, the complexity overhead of using BFS in each episode could be reduced, which is the purpose of the second model.

2) *Model 2 (MDRL With Demonstration Cloning)*: This model builds on the first model by introducing a novel concept: DC. The main idea here is to use an existing expert to *guide* the agents into collecting better experience. An expert is an existing agent that has expertise in tackling the same environment. In this work, a semi-expert is defined as an agent that has expertise in tackling a similar environment that slightly differs from the current one.

Reward Function: As discussed in Section IV-E, the reward function in (5) is computationally expensive, as it requires performing BFS in each episode. Alternatively, this model uses a sparse reward function given as

$$R_t = \begin{cases} -m + S, & \text{if target is localized} \\ -m, & \text{otherwise} \end{cases} \quad (6)$$

where S is a constant representing the sparse reward given to the agents if the target is found. This reward function maintains

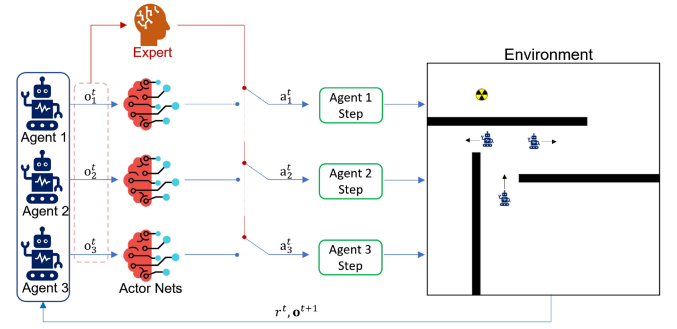


Fig. 8. MDRL with DC.

the same cost penalties as the ones in (5), but gives a large positive reward only when the target is localized.

Learning Process: The idea of using expert demonstrations with RL is derived from IL. In IL, agents learn to mimic the behavior of an expert by using expert demonstrations as a labeled data set in a supervised learning process, in which the aim is to reduce the loss between the agent and expert actions. Unlike RL, the concept of rewards does not exist in IL, as agents solely consider the expert as the main reference, without any input from their own experience. The work in [36] proposed using expert demonstrations with off-policy RL, where demonstrations are combined in the same buffer as agents' experiences and used to update the networks. Other works combine IL with off-policy [37] and on-policy [19] RL. In these works, the proposed models alternate between using the agent's own experience (i.e., rewards) and IL (i.e., mimicking an expert with no rewards). One issue in such works is the assumption that an expert, who is fully suited for the environment, exists. This limits the usability of such methods, as such experts may not always be available.

In this work, we propose MDRL-DC, a method that utilizes expert demonstrations while always maintaining the “learning from rewards” concept in RL. The proposed model alternates between RL and DC as follows: during RL, the agents explore the environment using the actions suggested by their own actor networks. During DC, the agents explore the environment based on the actions suggested by the expert. This way, while the expert guides the learning during DC, the agents are still learning based on their own experience, i.e., based on the rewards collected. This could result in better experiences, where the agents are more exposed to the sparse reward. This allows the use of semi-experts, i.e., experts that are familiar with similar environments but not necessarily the same environment. Bad actions suggested by the semi-expert will be identified through the collected rewards. Using the aforementioned works (RL + IL) with such semi-experts would not be efficient, since they are not fully suitable for the current environments, and agents during IL have no inputs in deciding whether an experience is good or not. As a result, the agents during the IL phase may learn what is against the learnt policy during the RL phase, resulting in a slower and potentially unstable learning.

Fig. 8 and Algorithm 2 illustrate and summarize the learning process of this model, which alternates between RL and

Algorithm 2 Training Process for the Second Model

Input: Initial actor and critic networks, Expert policy

```

1: while Step ≤ MaxNumOfSteps do:
2:    $\mathbf{o}^0 = \text{Env\_Reset}()$  #initial observations
3:   ExpertProb = rand() #generate a random probability
4:   for  $i = 0, 1, 2, \dots, \text{Episode\_Length}$ :
5:     for  $j = 1, 2, \dots, \text{Team\_Size}$ :
6:        $P^j = \text{actor}(\mathbf{o}_j^i)$  #get prob. dist. under actor network for agent  $j$ 
7:       if ExpertProb <  $E$ :
8:          $\mathbf{a}_e = \text{Expert}(\mathbf{o}_j^i)$  #get the action values
           from the expert, sorted from best to worst
9:          $\mathbf{a}_e[P^j[\mathbf{a}_e] < l] = 0$  #mask out actions that do not meet the
           threshold  $l$ .
10:         $\mathbf{a}_j^i = \max(\mathbf{a}_e)$ 
11:      else:
12:         $\mathbf{a}_j^i = \text{sample}(P^j)$  #sample an action from the actor distribution
13:    end for
14:     $\mathbf{a}^i = [\mathbf{a}_1^i, \mathbf{a}_2^i, \dots]$ 
15:     $\mathbf{o}^{i+1}, r^i, d^i = \text{Env\_Step}(\mathbf{a}^i)$ 
16:    Store  $\mathbf{o}^{i+1}$ ,  $\mathbf{a}^i$ , and  $r^i$  in their corresponding buffers
17:    Step = Step + 1
18:    if Step %  $H == 0$  then: #PPO update
19:      Compute the advantage estimate  $\hat{A}$ 
20:      Update the actor and critic networks using  $L^{CLIP+VF+S}(\theta)$ 
21:      Empty all buffers
22:    end if
23:    if  $d^i == 1$  then break #if episode terminates
24:  end for
25: end while

```

DC. At the beginning of each episode, an expert probability E determines whether the episode will follow RL or DC. During RL, the process is similar to that described in Algorithm 1, where agents act based on their copies of the actor network, and their experiences (observations, rewards, and idle flags) are buffered. When the expert is switched on, the expert takes the observations seen by the agents and suggests actions. These actions are then followed by the agents and the corresponding experiences are stored in the same buffers as the previous experiences. One key point here is that the actions suggested by the expert are *conditioned* on being probable under the agents' actor networks. This is because PPO is an on-policy algorithm, which means that the sampled actions need to follow the most recent version of the actor network. To handle this, a hyperparameter l is proposed, which represents the minimum probability needed, under the agent's own policy, for an action to be considered. In the proposed model, instead of suggesting the best action, the expert ranks all possible actions from best to worst, given higher values to better actions. An agent then takes the best action a , as suggested by the expert, that satisfies $P(a) > l$, where P is the probability distribution over the actions given by the agent's own policy. This way, the demonstrations given by the experts can help the agents get more exposed to rewards, while not violating the agents' own actor networks for on-policy learning. It is also worth mentioning that, despite using a fixed expert rate E throughout the learning, the effect of the expert is automatically annealed as the learning progresses. This is because the actor network becomes increasingly more confident, which results in less actions meeting the $P(a) > l$ condition.

It is assumed that there is access to the expert policy, and hence expert demonstrations can be generated online as needed. This is suitable for problems like target localization, where the expert could be previously trained on simpler environments (with no walls or with a single agent).

V. SIMULATION AND EVALUATION

This section presents several experiments conducted to evaluate the performance of the proposed methods. Simulations have been conducted using an Intel E5-2650 v4 Broadwell workstation equipped with 128-GB RAM, 800-GB SSD, and NVIDIA P100 Pascal GPU (16-GB HBM2 memory).

A. Simulation Environment

For all the experiments, a problem of radioactive target localization is used as an environment, where the aim is to localize a radioactive target of strength 1×10^9 photons/minute, in an area of size $1 \text{ km} \times 1 \text{ km}$. The data readings are generated using the principles of radiation physics, as explained in [8]. Assuming agents are carrying radiation detectors, the radiation readings at each detector follow a Poisson distribution that is given as [38]:

$$\text{CPM}_i \propto \frac{I}{d_i^2} \quad (7)$$

where CPM_i is the counts per minute at agent i 's detector, I is the source intensity in photons per minute, and d_i is the distance between agent i and the source. The background radiation is ignored since it is negligible when compared to the radioactive target. Additionally, the background radiation is assumed to be uniform throughout a given area, and hence it would not affect the target localization task which relies mainly on the difference between readings at different locations [8]. To simulate the effect of walls, the radioactive reading at an agent's sensor is attenuated by a factor of $\mu = 0.1$ for each wall between the agent and the radioactive target. For simplicity, it is assumed that all walls are similar and uniform, hence the attenuation factor is constant.

Although radiation target localization is used as an example to evaluate the proposed models, the applicability of the proposed system is valid for any target localization task, under the same assumptions of having a single target. This is due to the fact that a sensor reading in target localization tasks is given as a function of the distance between the agent and the target; the closer the agent is to the target, the higher the reading. For example, in several environmental applications, this idea is given as the Inverse Square Law, which states that a physical quantity at a certain location is inversely proportional to the square of the distance between the source of that physical quantity and that location. This is used to represent quantities, such as radiation, heat, and sound [22], [39].

For all the following experiments, each of the models was trained for 50 million steps. Each step represents a single interaction between the agents and the environment. In each training episode, the environment is reset with a randomized target and agents' locations, and with randomized walls placements and lengths. The number of walls is fixed for a single

TABLE I
HYPERPARAMETERS USED FOR PPO AND CAE TRAINING

PPO Hyperparameters	Value
Learning rate	3×10^{-4}
PPO clipping parameter ϵ	0.2
Entropy coefficient c_2	0.01
Discount factor γ	0.99
Discount factor λ	0.95
Timesteps per update (Horizon H)	4000
Number of epochs per update	20
CAE Hyperparameters	Value
Learning rate	1×10^{-3}
Embedding Size d	128
Dataset Size	100000
DC Hyperparameters	Value
Expert Rate E	0.2
Action Probability Threshold l	0.05

experiment. Walls here are always originating from the sides of the AoI. It is assumed that an agent cannot start in the same location as the target or another agent. An episode terminates when the target is found, or when a limit of 100 timesteps is reached. After every 40 000 training steps, the average results of 4000 testing steps, where the agents act greedily based on the latest policy update, are recorded and plotted. The list of hyperparameters used in training PPO, CAE, and DC is shown in Table I. The values for ϵ , c_2 , γ , and λ are set as suggested in the corresponding PPO work [21].

B. Performance Analysis

This section analyzes the performance of the proposed models in terms of episodic length and cost. As discussed in Section II-A, the main aim of the target localization methods is to achieve fast localization with low cost. The localization time is measured here by the episodic length, i.e., the number of steps it takes the agents from the beginning of the episode until the target is localized. The cost is given as the total number of movement steps taken by the agents, since a moving action reflects consumption of resources. In a single episodic step, the cost could be as low as 0 (no agents move) or N (all agents moved). At each step, each of the agents takes one of the nine actions discussed in Section IV-C. A moving action moves the agent 50 m in the chosen direction, after which another step (action) is required to carry on, based on newer observations. The maps dimensions ($m \times n$) are set to 30×30 , while the reduced dimensions are with a window size ($w \times w$) of 7×7 .

1) *Performance of MDRL-SR*: This section studies the performance of the MDRL-SR model in varying scenarios of team size and environment complexities. Specifically, the number of agents is varied from 1 to 4, while the number of walls is varied from 0 to 3. Figs. 9 and 10 show the episodic length and cost obtained throughout the learning process. As seen in all figures, the learning converges regardless of the team size or the environment complexity, with faster convergence in simpler environments. For all of these figures, the localization success rate is 100% at the end of the learning process, showing the effectiveness of the proposed approach. Intuitively, and as seen in the figures, longer episodes with higher cost are obtained as the environment gets more complex (more walls),

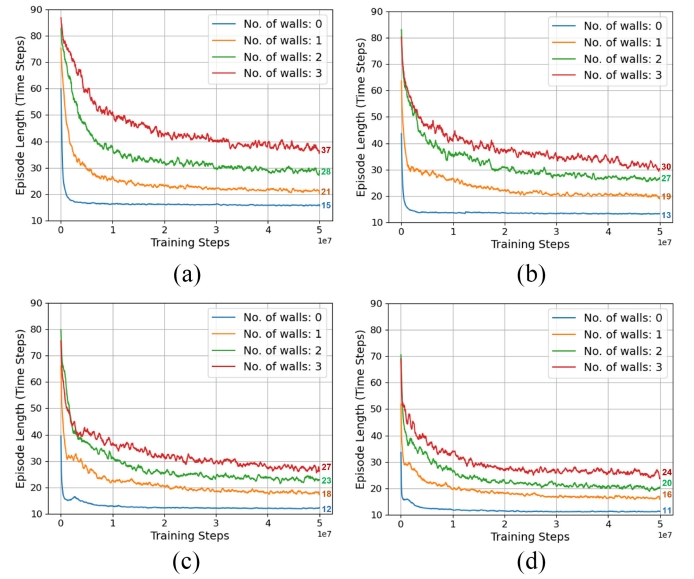


Fig. 9. Episodic length throughout the learning for a system of (a) one agent, (b) two agents, (c) three agents, and (d) four agents, for varying numbers of walls within the environment.

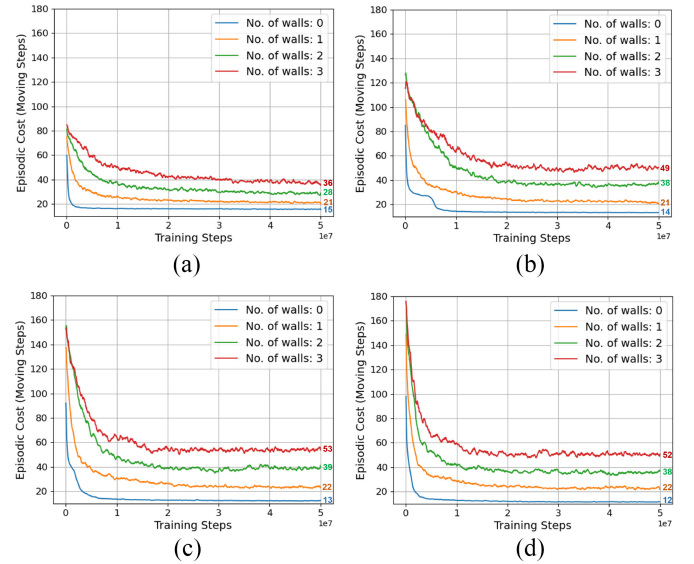


Fig. 10. Episodic cost throughout the learning for a system of (a) one agent, (b) two agents, (c) three agents, and (d) four agents, for varying numbers of walls within the environment.

since agents need more steps to navigate through the environment, and since more walls result in more attenuation of the radioactive readings, and hence harder localization.

As seen in Fig. 9, faster localization is achieved as the team size is increased from 1 [Fig. 9(a)] to 4 [Fig. 9(d)], for all environment complexity levels. On the other hand, the cost follows a slightly different behavior in Fig. 10. The episodic cost is affected by two factors: 1) the episode length and 2) the team size. Though having a bigger team results in more cost per step, on average, yet bigger teams result in faster localization, as per Fig. 9, and hence less total cost per episode. This tradeoff depends on the complexity of the problem. For example, in the case of simple environments with 0 or 1 wall,

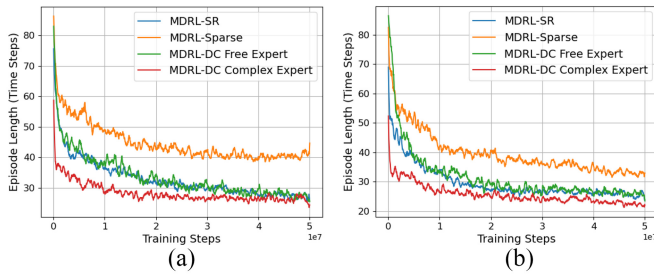


Fig. 11. Episodic time throughout the learning for a system of (a) three agents and (b) four agents, in an environment of three walls, for different learning models.

it can be seen that the cost is slightly reduced as the team size increases. That is, even when increasing the number of agents, the cooperation achieved by the agents results in faster localization at lower cost, as agents learn to prefer the IDLE action if they are not beneficial to the localization process (as discussed in Section II-A). For more complex environments (two or three walls), more exploration is needed since readings are significantly attenuated. As a result, increasing the number of agents could result in increasing the total cost, as seen when comparing Fig. 10(a) with Fig. 10(b) for the cases of two and three walls. This cost saturates at a certain team size, indicating that enough agents are available to handle the task and balance their resources without the need of unnecessary exploration. For example, for the case of two walls, the cost slightly increases from 20 to 38 when going from a team of one member to two members, then saturates around the same cost for higher team sizes. This saturation point happens at a later team size for a more complex environment of three walls, as the cost increases from 36 to 53 as the team size is increased from 1 to 3, and saturates for higher team sizes. This saturation, in all the aforementioned scenarios, is an indication that the agents have achieved good cooperation and resource management, even when the team size is increased.

C. Performance of MDRL-DC

This section analyzes the performance of the MDRL-DC model in comparison with the MDRL-SR model. This is done for teams of size three to four agents, in a complex environment of three walls. The performance of MDRL-DC with two different semi-experts is studied. The first is an expert in a multiagent free environment (no obstacles) while the second is an expert in a single-agent complex environment. The first expert is familiar with cooperation (and not complex environments), while the second expert is familiar with complex environments and not cooperation. Additionally, to show the drawback of using sparse rewards (without experts), the model MDRL-Sparse is included in the comparison. A sparse reward with the value $S = 50$ is used, as this was found to be sufficient to incentivize the agents.

Figs. 11 and 12 compare MDRL-SR with MDRL-Sparse and MDRL-DC in terms of episode length and cost. It is evident, when comparing MDRL-Sparse to the rest of the models, that using a sparse reward results in poor performance, which is due to the fact that agent is rarely exposed to the reward.

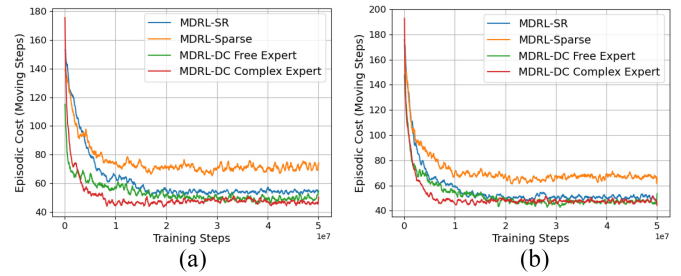


Fig. 12. Episodic cost throughout the learning for a system of (a) three agents and (b) four agents, in an environment of three walls, for different learning models.

It can also be seen that, when using a free expert, MDRL-DC is at least as good as MDRL-SR, if not better. This is significant since MDRL-DC is able to overcome the reward sparsity using expert demonstrations, and achieve a performance similar to a MDRL-SR that uses a shaped reward. Additionally, since the computationally expensive shaped reward is emitted in MDRL-DC, the learning process of 50M steps consume nearly 47% less wall-clock time when compared to MDRL-SR. When looking at the behavior of MDRL-DC with a free expert in Fig. 11, it can be seen that the performance is similar to MDRL-SR in terms of episode length. However, it can be seen in Fig. 12 that MDRL-DC with a free expert has a slightly better performance when compared to MDRL-SR, as it achieves faster convergence. This is attributed to the fact that the free expert is familiar with cooperation, and hence is capable of guiding the new agents into learning this attribute quicker. On the other hand, MDRL-DC with a complex expert slightly outperforming MDRL-SR in terms of time and cost, while maintaining the same 47% less wall-clock time for the learning process. This shows that, despite using a sparse reward and semi-experts that are not fully familiar with the same environments, the proposed DC method is able to achieve similar, if not better, performance to a system with shaped reward at almost half the runtime.

D. Benchmarks

This section compares the performance of MDRL-DC with several benchmarks. These benchmarks are either MDRL-based target localization or traditional target localization methods. In all of the following results, a simulation environment of three agents with a varying complexity of 1–3 walls is used to compare the performance in terms of episode length (time) and cost. The produced agents, following the training process, are placed in random environments for inference, where the episode length and cost is averaged over 1000 different episodes.

Fig. 13 compares the performance of MDRL-DC with three other deep RL models and two traditional methods, which are described as follows.

- 1) *ODMTL*: A MDRL approach for target localization [6] in free environments (no obstacles). This model ignores the existence of walls and attempts to localize the target without considering them in agents' observations.

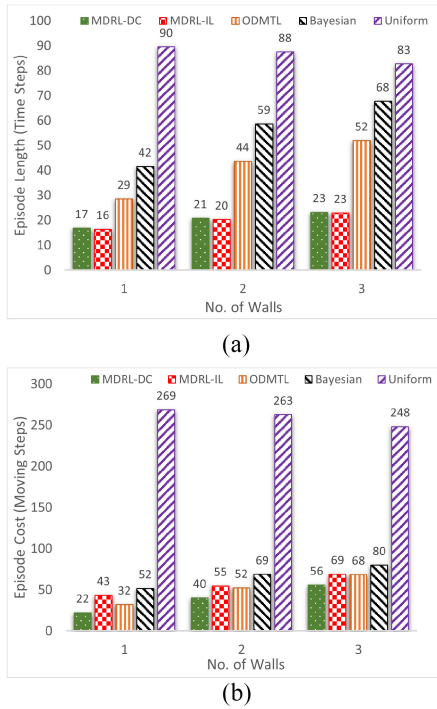


Fig. 13. Performance comparison between the benchmarks in terms of (a) episode length and (b) episode cost, throughout the learning process.

- 2) *MDRL-IL*: A MDRL model that alternates between RL and IL [19]. Unlike our proposed model, the agents in MDRL-IL during the IL stage blindly copy the behavior of the expert into their actor networks using behavioral cloning (BC), without having to experience any rewards in the environment.
- 3) *Bayesian*: A multiagent cooperative approach using Bayesian methods [8], [12]. In [12], a Bayesian framework is used to build and update local PDFs of the target location, which are shared between the agents to build a global PDF. This is used in an optimization objective that aims to find the target while balancing resources consumption between agents.
- 4) *Uniform*: A traditional method where each agent moves in a predefined path that uniformly covers the area [13].

As can be seen in the figure, the performance of the ODMTL model significantly falls behind the proposed MDRL-DC. Specifically, MDRL-DC has 50% less localization time and 24% less cost, on average, when compared to ODMTL. This shows the significance of considering walls/obstacles, as they have effects on sensor readings and agents planned paths. While both MDRL-IL and MDRL-DC use experts' demonstrations during the training, their performances are different. Here, both models use an expert that is familiar with single-agent complex environments. When analyzing Fig. 13(a), similar performance in terms of episodic length is noticed when comparing both models. However, when analyzing Fig. 13(b), the significance of MDRL-DC can be noticed. Since the expert is not familiar with cooperation, the agents in MDRL-IL struggle to cooperate. Even though these agents could learn cooperation during the RL stage (which is 80%

of the learning), this is overwritten by BC in the IL stage, since the expert always acts greedily and has no sense of the other agents. On the other hand, since the agents in MDRL-DC learn based on their own experience/rewards, they can identify through rewards if such experiences are good or bad, even if suggested by the expert. As a result, the model outperforms MDRL-IL, and shows its ability of using demonstrations from semi-experts. Specifically, the model achieves 32% less cost, on average, when compared to MDRL-IL.

When compared to traditional target localization methods, MDRL-DC shows dominance. Traditional methods could be efficient in handling free environments, but struggle with complex environments with walls, as such environments are difficult to define and represent. When compared to Bayesian methods, MDRL-DC localizes the target with 63% less time and 43% less cost, on average. On the other hand, uniform search methods do guarantee finding the target, since the agents scan the entire area, however this comes at huge costs, since all the agents are always active.

VI. CONCLUSION

In this article, the problem of target localization in complex environments is tackled, where two MDRL models are proposed. Agents' observations are modeled as 2-D heatmaps that capture essential information about their locations and readings, in addition to information about the environment complexity in terms of walls distribution. CAEs are used to create embeddings to represent walls, while CNNs are used for the actor-critic structures in the PPO algorithm that is used to optimize the learning. The first model, MDRL-SR, uses a shaped reward function, based on BFS, to guide the agents into cooperating to localize the target. The second model, MDRL-DC, builds on the first model by using a sparse reward instead, combined with a novel DC method that utilizes expert demonstrations to guide the learning of the new agents. MDRL-DC was found to achieve similar, if not better, performance as MDRL-SR despite using a sparse reward, and while consuming 47% less wall-clock time. MDRL-DC has also shown dominance when compared to existing benchmarks, including traditional localization methods and DRL-based methods. Specifically, MDRL-DC is found to achieve up to 63% less localization time and 43% less localization cost, when compared to traditional methods. When compared against works that combine reinforcement and IL, it was seen to achieve 32% less cost during the localization process, indicating its ability to produce better cooperative agents.

ACKNOWLEDGMENT

The extensive simulations in this research were enabled in part by support provided by Calcul Quebec (www.calculquebec.ca) and the Digital Research Alliance of Canada (alliancecan.ca).

REFERENCES

- [1] M. Abououf, R. Mizouni, S. Singh, H. Otrok, and E. Damiani, "Self-supervised online and light-weight anomaly and event detection for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 25285–25299, Dec. 2022.

- [2] A. Alagha, S. Singh, R. Mizouni, A. Ouali, and H. Otrok, "Data-driven dynamic active node selection for event localization in IoT applications—A case study of radiation localization," *IEEE Access*, vol. 7, pp. 16168–16183, 2019.
- [3] B. Yuan et al., "A UAV-assisted search and localization strategy in non-line-of-sight scenarios," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23841–23851, Dec. 2022.
- [4] O. M. Bushnaq, A. Chaaban, and T. Y. Al-Naffouri, "The role of UAV-IoT networks in future wildfire detection," *IEEE Internet Things J.*, vol. 8, no. 23, pp. 16984–16999, Dec. 2021.
- [5] Y. Wu and K. H. Low, "Route coordination of UAV fleet to track a ground moving target in search and lock (SAL) task over urban airspace," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 20604–20619, Oct. 2022.
- [6] A. Alagha, S. Singh, R. Mizouni, J. Bentahar, and H. Otrok, "Target localization using multi-agent deep reinforcement learning with proximal policy optimization," *Future Gener. Comput. Syst.*, vol. 136, pp. 342–357, Nov. 2022.
- [7] Z. Liu and S. Abbaszadeh, "Double Q -learning for radiation source detection," *Sensors*, vol. 19, no. 4, p. 960, 2019.
- [8] A. H. Liu, J. J. Bunn, and K. M. Chandy, "An analysis of data fusion for radiation detection and localization," in *Proc. 13th Int. Conf. Inf. Fusion*, 2010, pp. 1–8.
- [9] A. Sundaresan, P. K. Varshney, and N. S. V. Rao, "Distributed detection of a nuclear radioactive source using fusion of correlated decisions," in *Proc. IEEE 10th Int. Conf. Inf. Fusion*, 2007, pp. 1–7.
- [10] E.-W. Bai, K. Yosief, S. Dasgupta, and R. Mudumbai, "The maximum likelihood estimate for radiation source localization: Initializing an iterative search," in *Proc. IEEE 53rd IEEE Conf. Decis. Control*, 2014, pp. 277–282.
- [11] T. Lazna, P. Gabrlik, T. Jilek, and L. Zalud, "Cooperation between an unmanned aerial vehicle and an unmanned ground vehicle in highly accurate localization of gamma radiation hotspots," *Int. J. Adv. Robot. Syst.*, vol. 15, no. 1, pp. 1–19, 2018.
- [12] H. Xiao, R. Cui, and D. Xu, "A sampling-based Bayesian approach for cooperative multiagent online search with resource constraints," *IEEE Trans. Cybern.*, vol. 48, no. 6, pp. 1773–1785, Jun. 2018.
- [13] K. Ziocok and W. Goldstein, "The lost source, varying backgrounds and why bigger may not be better," in *Proc. AIP Conf.*, vol. 632, 2002, pp. 60–70.
- [14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [15] H. Sami, J. Bentahar, A. Mourad, H. Otrok, and E. Damiani, "Graph convolutional recurrent networks for reward shaping in reinforcement learning," *Inf. Sci.*, vol. 608, pp. 63–80, Aug. 2022.
- [16] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, Sep. 2020.
- [17] C. Berner et al., "Dota 2 with large scale deep reinforcement learning," 2019, *arXiv:1912.06680*.
- [18] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, "Deep reinforcement learning framework for autonomous driving," *Electron. Imag.*, vol. 29, no. 19, pp. 70–76, 2017.
- [19] G. Sartoretti et al., "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Autom. Lett.*, vol. 4, no. 3, pp. 2378–2385, Jul. 2019.
- [20] P. Proctor, C. Teuscher, A. Hecht, and M. Osinski, "Proximal policy optimization for radiation source search," *J. Nucl. Eng.*, vol. 2, no. 4, pp. 368–397, 2021.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [22] J.-C. Chin, D. K. Y. Yau, N. S. V. Rao, Y. Yang, C. Y. T. Ma, and M. Shankar, "Accurate localization of low-level radioactive source under noise and measurement errors," in *Proc. 6th ACM Conf. Embedded Netw. Sensor Syst.*, 2008, pp. 183–196.
- [23] S. K. Pandey and M. A. Zaveri, "Event localization in the Internet of Things environment," *Procedia Comput. Sci.*, vol. 115, pp. 691–698, 2017.
- [24] F. Grondin, I. Sobieraj, M. D. Plumbley, and J. R. Glass, "Sound event localization and detection using CRNN on pairs of microphones," in *Proc. Detection Classification Acoust. Scenes Events Workshop*, 2019, pp. 84–88.
- [25] Z. Liu, W. Dai, and M. Z. Win, "Node placement for localization networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2017, pp. 1–6.
- [26] A. Alagha, S. Singh, H. Otrok, and R. Mizouni, "RFLS-resilient fault-proof localization system in IoT and crowd-based sensing applications," *J. Netw. Comput. Appl.*, vol. 170, Nov. 2020, Art. no. 102783.
- [27] A. Alagha, R. Mizouni, S. Singh, H. Otrok, and A. Ouali, "SDRS: A stable data-based recruitment system in IoT crowdsensing for localization tasks," *J. Netw. Comput. Appl.*, vol. 177, Mar. 2021, Art. no. 102968.
- [28] A. Alagha, S. Singh, H. Otrok, and R. Mizouni, "Influence-and interest-based worker recruitment in crowdsourcing using online social networks," *IEEE Trans. Netw. Service Manag.*, early access, Oct. 27, 2022, doi: [10.1109/TNSM.2022.3217689](https://doi.org/10.1109/TNSM.2022.3217689).
- [29] M. Shurrah, S. Singh, R. Mizouni, and H. Otrok, "IoT sensor selection for target localization: A reinforcement learning based approach," *Ad Hoc Netw.*, vol. 134, Sep. 2022, Art. no. 102927.
- [30] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: A survey," *Artif. Intell. Rev.*, vol. 55, no. 2, pp. 895–943, 2022.
- [31] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Rep. (ICLR)*, 2016, pp. 1–2.
- [32] J. Sola and J. Sevilla, "Importance of input data normalization for the application of neural networks to complex industrial problems," *IEEE Trans. Nucl. Sci.*, vol. 44, no. 3, pp. 1464–1468, Jun. 1997.
- [33] Y. LeCun et al., "LeNet-5, Convolutional Neural Networks." 2015. [Online]. Available: <http://yann.lecun.com/exdb/lenet>
- [34] B. Baker et al., "Emergent tool use from multi-agent autocurricula," in *Proc. Int. Conf. Learn. Rep. (ICLR)*, 2020, pp. 1–6.
- [35] X. Lyu, Y. Xiao, B. Daley, and C. Amato, "Contrasting centralized and decentralized critics in multi-agent reinforcement learning," in *Proc. 20th Int. Conf. Auton. Agents MultiAgent Syst. (AAMAS)*, 2021, pp. 844–852.
- [36] M. Vecerik et al., "Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards," 2017, *arXiv:1707.08817*.
- [37] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Overcoming exploration in reinforcement learning with demonstrations," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 6292–6299.
- [38] G. F. Knoll, *Radiation Detection and Measurement*. New York, NY, USA: Wiley, 2010.
- [39] D. Davis and E. Patronis, *Sound System Engineering*. Hoboken, NJ, USA: CRC Press, 2014.



Ahmed Alagha received the B.Sc. and M.Sc. degrees in electrical and computer engineering from Khalifa University, Abu Dhabi, UAE, in 2016 and 2019, respectively. He is currently pursuing the Ph.D. degree with the Information System Engineering, Concordia University, Montreal, QC, Canada.

His research interests include multiagent systems, reinforcement learning, machine learning, artificial intelligence, IoT, sensing technologies, crowdsensing and sourcing, optimization methods, social networks, and radiation detection and localization.



Rabeb Mizouni received the M.Sc. and Ph.D. degrees in electrical and computer engineering from Concordia University, Montreal, QC, Canada, in 2002 and 2007, respectively.

She is currently an Associate Professor in electrical and computer engineering, Khalifa University, Abu Dhabi, UAE. She is interested in edge computing, crowdsensing/sourcing, blockchain, business process mining, machine learning, and cloud computing.

Dr. Mizouni is an Associate Editor in *IEEE Internet of Things Magazine* and a member of the Center for Cyber-Physical Systems at Khalifa University.



Jamal Bentahar received the Ph.D. degree in computer science and software engineering from Laval University, Quebec, QC, Canada, in 2005.

He is a Professor with Concordia Institute for Information Systems Engineering, Concordia University, Montreal, QC, Canada. From 2005 to 2006, he was a Postdoctoral Fellow with Laval University, and then NSERC Postdoctoral Fellow with Simon Fraser University, Greater Vancouver, BC, Canada. His research interests include computational logics, model checking, reinforcement and

deep learning, multiagent systems, and services computing.

Dr. Bentahar was an NSERC Co-Chair for Discovery Grants for Computer Science from 2016 to 2018.



Shakti Singh (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2000, 2003, and 2010, respectively.

He joined the faculty with the Electrical Engineering and Computer Science Department, Khalifa University of Science and Technology, Abu Dhabi, UAE, in 2010. He pursues research related to semiconductor devices and technologies, artificial intelligence, crowd sensing and sourcing, blockchain, IoT sensors and network, and UAV-based systems.



Hadi Otrok (Senior Member, IEEE) received the Ph.D. degree in ECE from Concordia University, Montreal, QC, Canada, in 2008.

He holds a Full Professor position with the Department of Electrical Engineering and Computer Science, Khalifa University, Abu Dhabi, UAE. Also, he is an Affiliate Associate Professor with the Concordia Institute for Information Systems Engineering, Concordia University and an Affiliate Associate Professor with the Electrical Department with Ecole de Technologie Supérieure, Montreal. His

research interests include the domain of blockchain, reinforcement learning, crowd sensing and sourcing, ad hoc networks, and cloud security.

Dr. Otrok Co-Chaired several committees at various IEEE conferences. He is also an Associate Editor of IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, *Ad-Hoc Networks* (Elsevier), and IEEE TRANSACTIONS ON SERVICE COMPUTING. He also served from 2015 to 2019 as an Associate Editor for IEEE COMMUNICATIONS LETTERS.